

# PARADIGMAS DE PROGRAMAÇÃO

## Exame de Preparação 2 — Reestruturado (Modelo 13,5 / 6,5)

UC: Paradigmas de Programação (PP)

Ano Letivo: 2025/2026

Duração: 2 horas

Material: Exame sem consulta

### Parte I — Aplicação Prática (Conceito do Trabalho)

13,5 valores

#### 1. Polimorfismo e Ligação Dinâmica em Java (3,5 valores)

Considere o seguinte trecho de código simplificado do teu domínio de veículos:

```
public class VehicleImpl {
    public String getDescricao() {
        return "Veiculo Geral";
    }
}

public class RefrigeratedVehicleImpl extends VehicleImpl {
    private double maxDistance;

    public RefrigeratedVehicleImpl(double maxDistance) {
        this.maxDistance = maxDistance;
    }

    @Override
    public String getDescricao() {
        return super.getDescricao() + " Refrigerado (Distancia: " + maxDistance + ")";
    }
}
```

Considerando a seguinte classe de teste:

```
public class TestVehicles {  
    public static void main(String[] args) {  
        VehicleImpl v1 = new VehicleImpl();  
        VehicleImpl v2 = new RefrigeratedVehicleImpl(30000.0);  
  
        System.out.println(v1.getDescricao());  
        System.out.println(v2.getDescricao());  
    }  
}
```

1. Qual é o output exato apresentado na consola pela execução de TestVehicles?
2. Explique detalhadamente o mecanismo de polimorfismo e ligação dinâmica (dynamic binding) de Java ao executar a chamada de v2.getDescricao().

## 2. Comparabilidade de Veículos em Java: Sobreposição do método equals() (3,5 valores)

Suponha que no teu projeto é agora necessário comparar veículos para evitar o registo de duplicados na instituição. Escreva as alterações necessárias na classe `VehicleImpl`, incluindo a redefinição do método `equals(Object obj)`.

Dois veículos são considerados iguais se, e apenas se, possuírem a mesma capacidade de carga e tipo de bem a recolher (`supplyType`). Garanta todas as proteções clássicas exigidas no contrato do `equals` em Java.

```
public class VehicleImpl implements Vehicle {  
    private final ItemType supplyType;  
    private final double maxCapacity;  
  
    public VehicleImpl(ItemType supplyType, double maxCapacity) {  
        this.supplyType = supplyType;  
        this.maxCapacity = maxCapacity;  
    }  
  
    @Override  
    public boolean equals(Object obj) {  
        // TODO: Implemente a sua resposta aqui  
    }  
}
```

### 3. Gestão de Rotas, Vetores Tradicionais e Métricas Logísticas (6,5 valores)

A instituição Cruz Vermelha necessita de uma classe de controle para gerir as rotas geradas pelo sistema. Escreva a classe `GestaoRotas`. **Não é permitida a utilização do Java Collections Framework (JCF).**

Implemente os métodos e atributos descritos abaixo, incluindo:

1. O vetor de armazenamento de rotas `routes` e o respetivo contador físico de posições.
2. O construtor que inicializa o vetor com uma capacidade física inicial de 3 posições.
3. O método `public boolean registarRota(Route r)`: adiciona uma rota ao vetor. Caso o vetor esteja cheio, deve redimensionar o mesmo manualmente para o dobro do tamanho. Garanta que a rota não é nula antes de a inserir. Retorna sucesso/insucesso.
4. O método `public double calcularDistanciaMediaPorTipo(ItemType tipo) throws RouteException`: calcula e retorna a distância média percorrida por todas as rotas cujos veículos transportam doações do tipo `tipo`. Se não existirem rotas compatíveis registadas, deve lançar a exceção **checked** `RouteException`.

```
public class GestaoRotas {  
    // TODO: Implemente os atributos, construtor, registarRota e calcularDistanciaMed:  
}
```

## Parte II — Conceitos Teóricos

6,5 valores

### 4. Perguntas de Escolha Múltipla (0,5 val. cada) (3,5 valores)

Selecione a única opção correta para cada uma das alíneas:

1. Considerando que A e E são classes em Java, e que B e D são interfaces, assinale a afirmação correta:
  - i. `interface F implements B, D {}`
  - ii. `interface F extends E {}`
  - iii. `interface F extends B, D {}`
  - iv. `class F implements E {}`
2. Em Java, a visibilidade `protected` garante acesso aos membros de uma classe por:
  - i. Qualquer classe exterior dentro do mesmo projeto.
  - ii. Apenas subclasses da mesma hierarquia (mesmo que noutro package) e por quaisquer classes pertencentes ao mesmo package.

- iii. Apenas classes derivadas diretas (subclasses diretas).
  - iv. Qualquer interface implementada por aquela classe.
3. O que acontece se tentarmos instanciar uma classe abstrata em Java?
- i. O compilador gera um erro de compilação.
  - ii. O programa executa normalmente mas inicializa todos os métodos a null.
  - iii. Lança uma RuntimeException em tempo de execução.
  - iv. O objeto é criado com sucesso, mas apenas os seus métodos estáticos funcionam.
4. Qual das seguintes é uma afirmação correta sobre o Encapsulamento em POO?
- i. Impede a herança de métodos entre classes sob o mesmo package.
  - ii. Garante que os atributos de uma classe podem ser lidos mas nunca modificados por nenhum método.
  - iii. Consiste na ocultação dos detalhes de implementação do estado de um objeto, controlando o seu acesso através de métodos públicos.
  - iv. Impede a redefinição de métodos em tempo de execução.
5. Em Java, uma classe pode:
- i. Estender múltiplas classes base ao mesmo tempo (herança múltipla).
  - ii. Implementar múltiplas interfaces e estender uma única classe.
  - iii. Implementar apenas uma interface de cada vez.
  - iv. Estender interfaces e implementar classes abstratas.
6. Uma classe declarada com a palavra-chave `final` em Java:
- i. Não pode ser instanciada usando o operador `new`.
  - ii. Não pode ser herdada/estendida por nenhuma outra classe.
  - iii. Não pode implementar nenhuma interface.
  - iv. Obrigatoriamente tem de ter todos os seus métodos declarados como finais.
7. Que tipo de herança suportam as interfaces Java?
- i. Nenhuma, as interfaces não suportam herança.
  - ii. Herança múltipla apenas a partir de classes abstratas.
  - iii. Herança múltipla a partir de outras interfaces utilizando a palavra-chave `extends`.
  - iv. Herança simples a partir de uma classe base.

## 5. Conceito de Mutabilidade de Objetos e Referências em Java (3,0 valores)

No teu projeto, ao registar novos veículos e devolver a lista através de `getVehicles()`, são gerados novos objetos (cópias). Explique que tipo de problema de segurança ocorreria se o método devolvesse a

referência direta do vetor privado interno (`return this.vehicles;`). Relacione a sua resposta com a mutabilidade de referências em Java.